

ARB assembly language

ARB assembly language is a low-level shading language, which can be characterized as an assembly language. It was created by the OpenGL Architecture Review Board (ARB) to standardize GPU instructions controlling the hardware graphics pipeline.

History

Texas Instruments created the first programmable graphics processor in 1985: the TMS34010, which allowed developers to load and execute code on the processor to control pixel output on a video display. This was followed by the TMS34020 and TMS34082 in 1989, providing programmable 3D graphics output.

Nvidia released its first video card NV1 in 1995, which supported quadratic texture mapping. This was followed by the Riva 128 (NV3) in 1997, providing the first hardware acceleration for Direct3D.

Various video card vendors released their own accelerated boards, each with their own instruction set for GPU operations. The OpenGL Architecture Review Board (ARB) was formed in 1992, in part to establish standards for the GPU industry.

The ARB and Nvidia established a number of OpenGL extensions to standardize GPU programming:^[1]

- EXT_texture_env_combine - provided a programmable method of combining textures.
- NV_register_combiners - GeForce 256
- NV_vertex_program - GeForce 3
- NV_texture_shader - GeForce 3
- NV_texture_shader3 - GeForce 4
- NV_vertex_program2 - GeForce FX
- NV_fragment_program - GeForce FX

This culminated with ARB's 2002 release of

- ARB_vertex_program^[2]
- ARB_fragment_program^[3]

These two extensions provided an industry standard for an assembly language that controlled the GPU pipeline for 3D vertex and interpolated pixel properties, respectively.

Subsequent high-level shading languages sometimes compile to this ARB standard. While 3D developers are now more likely to use a C-like, high-level shading language for GPU programming, ARB assembly has the advantage of being supported on a wide range of hardware.

Note however that some features, such as loops and conditionals, are not available in ARB assembly, and using them requires to adopt either the `NV_gpu_program4` extension, or the GLSL shading language.

All major graphics card manufacturers have supported ARB assembly language for years, since the NVIDIA GeForce FX series, the AMD R300-based cards (Radeon 9500 series and higher), and the Intel GMA 900.^[4] However, standard ARB assembly language is only at the level of Pixel Shader 2.0 and predates GLSL, so it has very few features. While nVidia has made proprietary extensions to ARB assembly languages that combine the fast compile speed of ARB assembly with modern OpenGL 3.x features, introduced with the GeForce 8 series, most non-nVidia OpenGL implementations do not provide the proprietary nVidia extensions to ARB assembly language^[5] and do not offer any other way to access all the shader features directly in assembly, forcing the use of GLSL even for machine generated shaders where assembly would be more appropriate.

Details

ARB_vertex_program

The ARB Vertex Program extension provides APIs to load ARBvp1.0 assembly instructions, enable selected programs, and to set various GPU parameters.

Vertex programs are used to modify vertex properties, such as position, normals and texture coordinates, that are passed to the next pipeline process: often a fragment shader; more recently, a geometry shader.

ARB_fragment_program

The ARB Fragment Program extension provides APIs to load ARBfp1.0 assembly instructions, enable selected programs, and to set various GPU parameters.

OpenGL fragments are interpolated pixel definitions. The GPU's vertex processor calculates all the pixels controlled by a set of vertices, interpolates their position and other properties and passes them onto its fragment process. Fragment programs allow developers to modify these pixel properties before they are rendered to a frame buffer for display.

OpenGL parameters

- **Attrib** parameters are per-vertex attributes such as vertex normals.
- **Local** parameters are applied across a program's entire data set for a given shader pass.
- **Env** parameters are applied across all programs.

ARB variables

All ARB assembly variables are float4 vectors, which may be addressed by xyzw or rgba suffixes.

Registers are scalar variables where only one element may be addressed.

- **ADDRESS** variables are registers.
- **ATTRIB** are per-vertex attributes.
- **PARAM** are uniform properties - constants, Env or Local.
- **TEMP** temporary variables.
- **ALIAS** provides alternate names for variables.
- **OUTPUT** designates variables that are passed back to the pipeline.

Vertex attributes

ARB assembly supports the following suffixes for vertex attributes:

- **position**
- **weight**
- **normal**
- **color**
- **fogcoord**
- **texcoord**
- **matrixindex**
- **attrib**

State matrices

ARB assembly supports the following state matrices:

- **modelview**
- **projection**
- **texture**
- **palette**
- **program**

The following modifiers may be used:

- **inverse**
- **transpose**
- **invtrans**

ARB assembly instructions

ARB assembly supports the following instructions:

- **ABS** - absolute value
- **ADD** - add
- **ARL** - address register load
- **DP3** - 3-component dot product
- **DP4** - 4-component dot product

- **DPH** - homogeneous dot product
- **DST** - distance vector
- **EX2** - exponential base 2
- **EXP** - exponential base 2 (approximate)
- **FLR** - floor
- **FRC** - fraction
- **LG2** - logarithm base 2
- **LIT** - compute light coefficients
- **LOG** - logarithm base 2 (approximate)
- **MAD** - multiply and add
- **MAX** - maximum
- **MIN** - minimum
- **MOV** - move
- **MUL** - multiply
- **POW** - exponentiate
- **RCP** - reciprocal
- **RSQ** - reciprocal square root
- **SGE** - set on greater than or equal
- **SLT** - set on less than
- **SUB** - subtract
- **SWZ** - extended swizzle
- **TEX** - Texture lookup
- **XPB** - cross product

This is only a partial list of assembly instructions; a reference can be found here : [Shader Assembly Language \(ARB/NV\) Quick Reference Guide for OpenGL \(http://www.renderguild.com/gpuguide.pdf\)](http://www.renderguild.com/gpuguide.pdf).

ARB assembly provides no instructions for flow control or branching. SGE and SLT may be used to conditionally set or clear vectors or registers.

ARB interfaces provide no compiling step for assembly language.

GL_NV_fragment_program_option extends the ARB_fragment_program language with additional instructions. GL_NV_fragment_program2, GL_NV_vertex_program2_option and GL_NV_vertex_program3 extend it further.

Sample code

A sample trivial ARB vertex shader

```
!!ARBvp1.0
TEMP vertexClip;
DP4 vertexClip.x, state.matrix.mvp.row[0], vertex.position;
DP4 vertexClip.y, state.matrix.mvp.row[1], vertex.position;
```

```
DP4 vertexClip.z, state.matrix.mvp.row[2], vertex.position;
DP4 vertexClip.w, state.matrix.mvp.row[3], vertex.position;
MOV result.position, vertexClip;
MOV result.color, vertex.color;
MOV result.texcoord[0], vertex.texcoord;
END
```

A sample trivial ARB fragment shader

```
!!ARBfp1.0
TEMP color;
MUL color, fragment.texcoord[0].y, 2.0;
ADD color, 1.0, -color;
ABS color, color;
ADD result.color, 1.0, -color;
MOV result.color.a, 1.0;
END
```

See also

- [OpenGL Architecture Review Board](#)
- [Shading languages](#)
- [SPIR-V](#)
- [Comparison of assemblers](#)

References

1. Simon Green. "History of Programmability in OpenGL" (http://download.nvidia.com/developer/presentations/2004/6800_Leagues/6800_Leagues_OpenGL_exts.pdf) (PDF).
2. ARB. "[ARB_vertex_program](https://www.khronos.org/registry/OpenGL/extensions/ARB/ARB_vertex_program.txt)" (https://www.khronos.org/registry/OpenGL/extensions/ARB/ARB_vertex_program.txt).
3. ARB. "[ARB_fragment_program](https://www.khronos.org/registry/OpenGL/extensions/ARB/ARB_fragment_program.txt)" (https://www.khronos.org/registry/OpenGL/extensions/ARB/ARB_fragment_program.txt).
4. Wildfire Games. "[List of graphics cards and OpenGL renderers that support ARB assembly language](http://feedback.wildfiregames.com/report/opengl/feature/GL_ARB_fragment_program)" (http://feedback.wildfiregames.com/report/opengl/feature/GL_ARB_fragment_program).
5. Wildfire Games. "[List of graphics cards and OpenGL renderers that support nVidia's extensions to ARB assembly language](http://feedback.wildfiregames.com/report/opengl/feature/GL_NV_gpu_program4)" (http://feedback.wildfiregames.com/report/opengl/feature/GL_NV_gpu_program4).

Retrieved from "https://en.wikipedia.org/w/index.php?title=ARB_assembly_language&oldid=1312176532"